

AGENTS.md

Status: Mandatory project-wide engineering and security policy

Applies to: Human engineers, coding agents, automated workflows, CI/CD, services, infrastructure, data pipelines, APIs, and AI-enabled features

Owner: Engineering and Security

Default decision: When a proposed change conflicts with this document, stop the change and obtain a documented exception before continuing.

Operating principles

All contributors must protect confidentiality, integrity, availability, privacy, safety, and user trust. Work must be **correct, reviewable, testable, observable, reversible, and secure by default**. Prefer the smallest change that solves the stated problem, avoid speculative features, and document material assumptions.

SDAIA describes its generative-AI guidance as covering adoption, common scenarios, challenges, responsible-use principles, and recommended practices. ¹ The governance rules below apply those principles to engineering work involving AI; they supplement, and do not replace, applicable law, regulation, contracts, or organizational policy.

Engineering Rules

1. Scope and change discipline

1. Read the repository instructions, relevant architecture, existing tests, and adjacent implementation before changing code.
2. Preserve established project conventions unless there is a documented reason to change them.
3. Keep changes focused. Do not mix unrelated refactors, formatting changes, dependency upgrades, or behavior changes into one patch.
4. Do not remove, weaken, bypass, or disable a security control, test, audit log, validation rule, or approval gate merely to make a change pass.
5. Record significant design decisions, assumptions, trade-offs, migrations, compatibility impacts, and operational requirements.
6. Prefer reversible changes and include a rollback or disablement path for high-risk features.

2. Code quality and maintainability

1. Use clear names, small cohesive functions, explicit interfaces, and predictable control flow.
2. Make invalid states difficult to represent. Use strict types, schemas, invariants, and domain validation where available.
3. Avoid hidden global state, duplicated business rules, unnecessary abstractions, and premature optimization.
4. Preserve backward compatibility unless a breaking change is explicitly approved and documented.
5. Handle errors deliberately. Do not silently swallow failures, return misleading success responses, or use catch-all fallbacks that hide defects.
6. Keep public APIs, configuration, migrations, and contracts documented and versioned where appropriate.

3. Testing and verification

1. Add or update tests for every behavior change, including success paths, failure paths, boundary conditions, authorization boundaries, and regressions.
2. Run the narrowest relevant tests first, then the complete required suite before delivery.
3. Test integrations at their real trust boundaries. Mocks must not conceal serialization, authentication, timeout, retry, or permission failures.
4. Use deterministic tests. Control time, randomness, external dependencies, and test data where practical.
5. Do not weaken assertions, skip failing tests, or increase timeouts without recording the reason and follow-up action.
6. Verify migrations against realistic data volumes and provide a safe rollback or recovery plan.
7. Confirm observability, performance, accessibility, and security requirements for user-facing or production-critical changes.

4. Dependencies and supply chain

1. Add dependencies only when they provide clear value and are compatible with project licensing and security requirements.
2. Pin and verify dependencies with lockfiles, integrity checks, trusted registries, and an approved update process.

3. Review dependency ownership, maintenance, permissions, data handling, licensing, vulnerabilities, and removal options.
4. Run appropriate static analysis, dependency, secret, container, infrastructure-as-code, and software-composition scans in CI/CD.
5. Use reproducible builds, protected build runners, restricted deployment identities, and signed or provenance-tracked artifacts where supported.

5. Data and API contracts

1. Define schemas and validate data at system boundaries.
2. Version public or shared contracts and document compatibility behavior.
3. Use explicit timeouts, bounded retries with backoff, idempotency controls, pagination, request-size limits, and rate limits.
4. Treat all external data and service responses as untrusted until validated.
5. Make data ownership, retention, deletion, classification, and permitted use explicit.
6. Do not expose internal implementation details through APIs, client bundles, logs, or error messages.

6. Operations and delivery

1. Configuration must be environment-specific, reviewed, reproducible, and safe by default.
2. Production changes require an identified owner, deployment plan, monitoring plan, rollback path, and incident contact where risk warrants it.
3. Use feature flags or staged rollout for high-risk changes, and remove obsolete flags after stabilization.
4. Keep runbooks, health checks, alerts, dashboards, backup procedures, and restoration tests current.
5. Monitor latency, errors, saturation, availability, security events, and cost for production-critical paths.
6. Document manual operational actions and ensure they are authorized, auditable, and reversible where possible.

7. Code-review gate

Every material change must answer the following questions before approval:

Review area	Required question
-------------	-------------------

Correctness	Does the implementation satisfy the requested behavior and preserve existing contracts?
Failure behavior	What happens on invalid input, dependency failure, timeout, partial completion, and retry?
Security	Are secrets, authorization, input validation, output encoding, privacy, and abuse cases addressed?
Testing	Are meaningful positive, negative, boundary, regression, and integration tests present?
Operations	Can the change be monitored, rolled back, migrated, and supported in production?
Maintainability	Is the change understandable, minimal, and consistent with the repository?
AI governance	If AI is involved, are provenance, human oversight, data controls, and output validation documented?

Security Rules

1. Secrets and sensitive configuration

1. **Never expose secrets.** Do not commit API keys, passwords, private keys, access tokens, session tokens, certificates, or connection strings to source code, logs, tickets, screenshots, artifacts, or public repositories.
2. **Always use environment variables or an approved secret manager.** Secrets must be injected at runtime and must not be stored in client-side bundles or checked-in configuration files.
3. **Never print secrets.** Redact tokens, authorization headers, cookies, personal data, and credentials from logs and error reports.
4. Use short-lived, scoped credentials, role-based access, workload identity, token rotation, and least privilege.
5. If a secret is exposed, revoke it, replace it, investigate access history, and preserve relevant evidence immediately.

2. Input, output, database, and command safety

1. **Always validate user input on the server.** Client-side validation is not a security control.
2. Use strict schemas, allowlists, length and size limits, canonicalization, and context-aware output encoding.
3. **Never generate insecure SQL.** Use parameterized queries, prepared statements, or a safely configured ORM for all variable data.
4. Never concatenate untrusted input into SQL, shell commands, LDAP filters, template expressions, or regular expressions.
5. Treat uploaded files, external responses, webhooks, and third-party data as untrusted; validate, limit, scan, and isolate them.
6. Apply database least privilege, tenant isolation, query limits, encryption, and audit logging without recording sensitive values.

3. Authentication, authorization, and sessions

1. Authenticate every protected request and authorize every sensitive action server-side.
2. Never trust hidden UI controls, client-provided roles, unverified headers, or model-generated permission claims.
3. **Never store passwords in plain text.** Use a modern adaptive password-hashing algorithm with a unique salt and safe work factor.
4. Protect sessions with secure, HttpOnly, appropriately scoped cookies; rotate identifiers after authentication or privilege changes; expire revoked and inactive sessions.
5. Require multi-factor authentication for administrators and privileged operations unless a documented exception is approved.
6. Fail closed: if identity, policy, or permission evaluation fails, deny access.

4. Errors, logs, monitoring, and privacy

1. **Never reveal stack traces.** Return a generic error and correlation identifier to users; keep diagnostics in protected internal telemetry.
2. Do not expose database names, filesystem paths, library versions, internal hostnames, tokens, or policy details.
3. Log security-relevant events with timestamp, service, action, outcome, identity, resource, and correlation ID while applying data minimization.
4. Protect logs with access controls, retention limits, encryption where required,

structured logging, and injection prevention.

5. Collect and retain only necessary data. Classify it, define its purpose and retention, encrypt it in transit and at rest, and restrict decryption access.
6. Do not place confidential or personal data into public AI tools or unapproved third-party services.

SDAIA Governance Rules for AI-Enabled Systems

SDAIA's official AI materials explain that generative-AI guidance is intended to support responsible adoption and use, identify challenges and considerations, and provide recommended practices for common scenarios. ¹ The following controls are mandatory for any feature that uses generative AI, machine learning, retrieval, autonomous agents, or model-generated actions.

1. Governance, accountability, and risk classification

1. Assign a named business owner, technical owner, data owner, and security or privacy reviewer before production use.
2. Record the system purpose, intended users, affected parties, model/provider, data sources, limitations, prohibited uses, risk level, and approval status.
3. Apply stronger review, testing, monitoring, and human-approval requirements to systems that affect legal rights, employment, healthcare, finance, safety, access, identity, or significant personal outcomes.
4. Maintain an inventory of models, prompts, tools, retrieval indexes, datasets, providers, versions, and material changes.
5. Review the system periodically and whenever the model, provider, data, prompt, tool permissions, target users, or decision impact changes.

2. Responsible data use and privacy

1. Use only data that is necessary, authorized, appropriately classified, and permitted for the stated purpose.
2. Do not use personal, confidential, regulated, proprietary, or restricted data for training, prompting, retrieval, evaluation, or monitoring unless approved safeguards are in place.
3. Mask or pseudonymize data for development and testing; prevent cross-tenant retrieval and accidental memorization.
4. Define retention and deletion for prompts, outputs, embeddings, chat history, uploaded files, evaluation records, and provider-side data.

5. Inform users when content is generated or materially assisted by AI, explain relevant limitations, and provide a correction or escalation path.

3. Human oversight and accountability

1. AI output must not be the sole basis for high-impact decisions. A qualified human must be able to review, override, correct, and document the outcome.
2. Require explicit human confirmation before irreversible, high-impact, external, financial, privileged, or safety-sensitive actions.
3. Preserve an audit trail of material prompts, model version, retrieved sources, tool calls, output, reviewer, decision, and override where lawful and appropriate.
4. Provide a clear owner for incidents, complaints, harmful outputs, bias reports, privacy requests, and model-performance degradation.

4. Safety, security, and technical controls

1. Treat prompts, retrieved documents, model outputs, tool results, and conversation history as untrusted data.
2. Defend against direct and indirect prompt injection by separating instructions from data, validating retrieved content, constraining tools, and enforcing application policy outside the model.
3. **Never allow model output to directly execute code, SQL, shell commands, browser actions, or privileged API calls.** Use allowlists, deterministic validation, sandboxing, scoped credentials, rate limits, and approval gates.
4. Validate outputs for format, factuality, harmful content, privacy leakage, bias, policy compliance, authorization, and prompt leakage before display or action.
5. Protect models and data against poisoning, unauthorized extraction, memorization, denial of service, unbounded cost, and data exfiltration.
6. Fail safely when the model is unavailable, uncertain, inconsistent, or outside its approved scope; do not invent confidence or silently substitute an unsafe behavior.
7. Test realistic abuse cases, including jailbreaks, prompt injection, sensitive-data extraction, unsafe tool use, cross-tenant leakage, malformed inputs, and adversarial documents.

5. Transparency, quality, and continuous monitoring

1. Distinguish generated content from verified facts and avoid presenting unverified output as authoritative.

2. Measure accuracy, groundedness, harmful-output rate, refusal behavior, latency, availability, cost, drift, fairness indicators, and security incidents as applicable.
3. Use approved evaluation datasets that are representative, privacy-safe, versioned, and protected from contamination.
4. Monitor production behavior and establish thresholds that trigger human review, feature disablement, rollback, provider change, or incident response.
5. Preserve model and data provenance, including provider, model version, prompt/template version, retrieval sources, safety configuration, and reviewer.

Pull-Request Checklist

- ☐ The change is focused, documented, tested, observable, and reversible where risk warrants it.
- ☐ No secrets, credentials, personal data, or confidential content are committed or logged.
- ☐ All external input is validated server-side and output is safely encoded.
- ☐ All database access uses parameterized queries or a safe equivalent.
- ☐ Authentication and authorization are enforced on every protected operation.
- ☐ Errors do not reveal stack traces or internal implementation details.
- ☐ Dependencies and build artifacts were scanned and reviewed.
- ☐ Sensitive data is minimized, classified, encrypted, and handled only by approved services.
- ☐ AI outputs and tool calls are constrained, validated, monitored, and subject to human oversight when risk is high.
- ☐ SDAIA governance requirements were assessed, documented, and approved for AI-enabled changes.
- ☐ Any exception is documented, approved, assigned to a risk owner, and time-limited.

Security Exception Format

Any exception must identify the affected rule, business justification, risk assessment, risk owner, compensating controls, expiration date, monitoring plan, and remediation deadline. Exceptions are temporary and must be reviewed before renewal.

References

- [1] SDAIA — Artificial Intelligence and Generative AI Guideline overview
- [2] SDAIA — Generative AI guide announcement
- [3] SDAIA — Generative AI Guideline for Public use